

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import (
    confusion_matrix,
    accuracy_score,
    precision_score,
    recall_score,
    classification_report)
```

1. Load Dataset

```
df = pd.read_csv("/content/diabetes.csv")

print("First 5 rows:")
print(df.head())
print("\nDataset shape:", df.shape)
```

First 5 rows:

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	\
0	6	148	72	35	0	33.6	
1	1	85	66	29	0	26.6	
2	8	183	64	0	0	23.3	
3	1	89	66	23	94	28.1	
4	0	137	40	35	168	43.1	

	Pedigree	Age	Outcome
0	0.627	50	1
1	0.351	31	0
2	0.672	32	1
3	0.167	21	0
4	2.288	33	1

Dataset shape: (768, 9)

2. Check for missing / zero values

```
print("\nMissing values per column:")
print(df.isnull().sum())
zero_invalid_cols = ["Glucose", "BloodPressure", "SkinThickness", "Insulin", "BMI"]

for col in zero_invalid_cols:
    df[col] = df[col].replace(0, np.nan)
    df[col] = df[col].fillna(df[col].mean())
```

Missing values per column:

Pregnancies	0
Glucose	0
BloodPressure	0
SkinThickness	0
Insulin	0
BMI	0
Pedigree	0
Age	0
Outcome	0

dtype: int64

3. Split features and target

```
X = df.drop("Outcome", axis=1)
y = df["Outcome"]
```

4. Train/test split

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.30,
    random_state=42,
    stratify=y
)

print("\nTraining set:", X_train.shape)
print("Testing set:", X_test.shape)
```

```
Training set: (537, 8)
Testing set: (231, 8)
```

5. Feature scaling (important for distance-based KNN)

```
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

6. Train KNN model

```
k = 7
knn = KNeighborsClassifier(n_neighbors=k)
knn.fit(X_train, y_train)
```

```
▼ KNeighborsClassifier ⓘ ?
KNeighborsClassifier(n_neighbors=7)
```

7. Predict

```
y_pred = knn.predict(X_test)
```

8. Evaluation metrics

```
cm = confusion_matrix(y_test, y_pred)
accuracy = accuracy_score(y_test, y_pred)
error_rate = 1 - accuracy
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)

print("\nConfusion Matrix:")
print(cm)
```

```
Confusion Matrix:
[[125  25]
 [ 35  46]]
```

```
print(f"\nAccuracy:  {accuracy:.4f} ({accuracy*100:.2f}%)")
print(f"Error Rate:  {error_rate:.4f} ({error_rate*100:.2f}%)")
print(f"Precision:  {precision:.4f}")
print(f"Recall:      {recall:.4f}")

print("\nFull classification report:")
print(classification_report(y_test, y_pred))
```

```
Accuracy:  0.7403 (74.03%)
Error Rate: 0.2597 (25.97%)
Precision:  0.6479
Recall:     0.5679
```

```
Full classification report:
              precision    recall  f1-score   support

     0           0.78       0.83       0.81       150
     1           0.65       0.57       0.61        81

   accuracy          0.74          0.74          0.74       231
  macro avg          0.71       0.70       0.71       231
 weighted avg          0.73       0.74       0.74       231
```

Start coding or [generate](#) with AI.

